

A Million Combinator Reductions a Second: Hardware-Accelerated Evolution of SKI Programs on Cloud FPGAs

Mark Santolucito¹, Christian Scaff¹, Andrew Bonilla¹, and Zenas Boamah¹

¹Barnard PL Labs, Barnard College, Columbia University, USA
msantolu@barnard.edu

Abstract

Evolutionary search over *computation* — programs, circuits, reaction networks — is a recurring theme in artificial life, but it is chronically bottlenecked by the cost of *evaluating* candidates. We present a massively-parallel evolutionary substrate that evolves SKI combinatory-logic programs entirely in FPGA fabric: each soft core generates, reduces (to weak head normal form via in-place graph reduction in block RAM), and scores candidates against a target function with no host in the inner loop. On a single low-cost ECP5 FPGA we measure **>1.0 million candidate evaluations per second** across 16 parallel cores, scaling linearly with core count, and synthesis shows ~ 60 cores fit on the device (a projected ~ 3.8 M eval/s). Against a bit-identical C implementation on a modern CPU (Apple M4), the FPGA matches a full chip’s throughput at roughly **one-twelfth the power**, and the architecture projects to ~ 45 datacenter-FPGA cards matching a 100k-CPU farm at $\sim 80\times$ lower power. The system is built and run entirely through **Manhattan Reasoning Gym (MRG)**, a cloud FPGA platform that lets researchers submit HDL and drive hardware over a simple SDK without owning boards — lowering the barrier to hardware-accelerated artificial-life experiments. All designs and benchmarks are open.

Submission type: **Late Breaking Abstracts**

Data/Code available at: <https://manhattanreasoning.com>

Motivation

Artificial life has a long tradition of *evolving computation*: Tierra’s self-replicating machine code (Ray, 1991), Fontana & Buss’s AIChem (an algorithmic chemistry over λ -terms) (Fontana and Buss, 1994), and the broader thread of algorithmic chemistries in which programs are the molecules and reduction is the reaction. SKI combinatory logic is an especially clean substrate for this lineage — three primitives (S , K , I), one operation (application), Turing-complete, and free of variable binding, so terms are pure trees that mutate and recombine trivially.

©2026 The Authors. Published under a Creative Commons Attribution 4.0 International (CC BY 4.0) license.

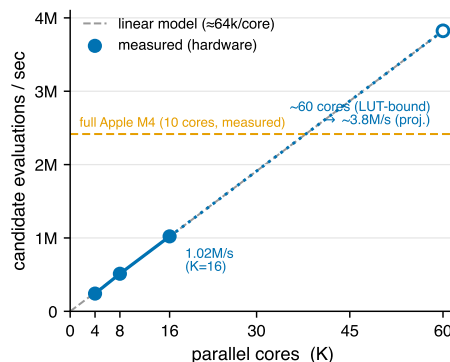


Figure 1: Throughput scales linearly with on-chip cores. Three measured points ($K = 4, 8, 16$) lie on a ~ 63.7 k-eval/s/core line; synthesis bounds the device at ~ 60 cores (LUT-limited), projecting ~ 3.8 M eval/s on one $\sim \$40$, ~ 3 W ECP5-85. The all-core Apple M4 is shown for reference; the FPGA passes a full modern CPU near 38 cores.

The persistent obstacle is **evaluation throughput**: evolutionary and open-ended searches are dominated by the cost of reducing candidates and scoring them, and richer experiments are simply unaffordable on CPUs at the scales ALIFE questions often demand. We ask: *what becomes possible if candidate evaluation is 10^6 – 10^8 /sec?* Our answer is to move the entire evolutionary inner loop — generation, reduction, fitness — onto FPGA fabric, replicate it, and expose it through a cloud platform usable without hardware.

System

Substrate. A candidate is a pure SKI term, stored as a graph of 32-bit nodes in block RAM (tag + two child pointers). The three rewrite rules are $I x \rightarrow x$, $K x y \rightarrow x$, and $S x y z \rightarrow x z (y z)$.

On-chip evolutionary core. Each soft core runs autonomously: (1) *Generation* — an LFSR drives a generator that writes a random *well-formed* term: every application node references only earlier nodes, so terms are acyclic and in-range by construction. (2) *Reduction* —

a weak-head-normal-form graph reducer unwinds the left spine with a 3-deep sliding window (SKI’s widest rule needs only three arguments, so no unbounded stack), rewrites the head redex in place to preserve sharing, and guards against non-termination with a per-candidate step cap. (3) *Fitness* — boolean targets use Church encodings ($\text{TRUE} = K$, $\text{FALSE} = KI$), and the output is read by applying the candidate to inert sentinel atoms. Fitness is the number of truth-table rows matched; divergent or malformed candidates score zero by construction.

Parallel engine. K cores share one Wishbone control interface; the host writes the target and a seed, then reads a global candidate counter and the best fitness found. There is **no host traffic in the inner loop** — the cores share nothing, so throughput tracks core count.

Platform. The whole thing is developed, built (Amaranth $\rightarrow$ Yosys $\rightarrow$ nextpnr-ECP5 $\rightarrow$ bitstream), flashed, and driven through **Manhattan Reasoning Gym**: HDL is submitted over an SDK, programmed onto an idle cloud FPGA, and controlled with simple register reads/writes. No board ownership, no local toolchain.

Results

All measurements: evolving 2-input XOR (`cand.size=16`, step cap 2000), ~ 5 s free-running windows, on an ECP5-85 (LFE5UM5G-85F, 50 MHz, $\approx \$40$, ≈ 3 W).

K cores	cand./sec	per core	XOR
4	241,670	60,417	solved 4/4
8	512,152	64,019	solved 4/4
16	1,021,698	63,856	solved 4/4

Table 1: Throughput scales linearly with cores; the search solves the target at every K . Going $4 \rightarrow 16$ cores yields $4.23\times$ throughput at a flat $\sim 63\text{k/s}$ per core — the cores are genuinely independent.

Resource scaling (yosys synth_ecp5). $\sim 1,250$ LUT-equiv, 1 BRAM, 2 DSP per core, linear in K ; LUTs bind first, so ~ 60 **cores fit** on the ECP5-85. ($K = 16$ is verified to route on real silicon; place-and-route (~ 15 min) is the practical cost at high K , not fabric.)

Versus CPUs (bit-identical workload). A C port (same generation, reducer, and evaluation) on an Apple M4 measures $\sim 320\text{k eval/s}$ per core, $\sim 2.42\text{ M/s}$ across the full chip (~ 22 W). Per *core* the CPU wins $\sim 5\times$ — purely clock ($\approx 88\times$ the 50 MHz SoC) — but the FPGA core is $\sim 18\times$ **more cycle-efficient** (~ 780 vs $\sim 14,000$ cycles/candidate): the datapath *is* the reducer. Net: one ~ 3 W ECP5 matches a 3 nm flagship’s whole-chip throughput at $\sim 12\times$ **better eval/sec/W** (Figure 2). Projected to datacenter FPGAs, ~ 45 **cards match a 100k-CPU farm at $\sim 80\times$ lower power and $\sim 40\text{--}80\times$ lower cost.**

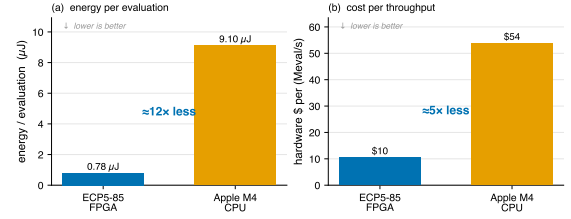


Figure 2: Total cost of ownership vs a modern CPU (bit-identical workload); both panels are clock-rate-independent cost-per-work, so lower is better. (a) Energy per evaluation: $\sim 12\times$ less (~ 0.78 vs $\sim 9.1\ \mu\text{J}$). (b) Hardware \$ per sustained Meval/s: $\sim 5\times$ less. Power ($\sim 3/\sim 22$ W) and chip cost ($\sim \$40/\sim \130) are rough estimates; the gaps are robust to $\sim 2\times$ uncertainty.

Why it works — and where it doesn’t. The advantage is greatest when candidates are *small* (short terms, shallow truth tables, few reductions). Hard monolithic targets (e.g. a 12-bit adder: 2^{24} rows) are not a throughput problem but a *search-strategy* one; the response is decomposition — evolve a small reusable cell, compose structurally.

Future Work

The reusable asset is the **engine architecture** — generate $\rightarrow$ evaluate-on-fabric $\rightarrow$ select, massively parallel — not the SKI substrate specifically. Two directions.

Combinator programs to circuits (evolvable hardware). Swap the SKI genome for the configuration vector of a *virtual reconfigurable circuit* — a soft, runtime-programmable LUT/gate array (CGP-style (Miller, 2011)). A candidate loads in microseconds (no resynthesis) and is evaluated as *real* behavior on the fabric, so fitness can include actual latency, switching activity, and active-cell count: the intrinsic-evolvable-hardware tradition (Thompson, 1996) at newly affordable evaluation rates.

ML-relevant primitives, with quality-diversity. ML’s tolerance for approximation smooths the landscape and opens a large design space (approximate MACs (Vaříček and Sekanina, 2015), stochastic-computing elements, systolic tiles). MAP-Elites search (Mouret and Clune, 2015) would *illuminate* an accuracy-vs-area-vs-power map rather than return a single winner — a natural fit for open-ended ALIFE methods, and a path to evolving hardware that accelerates ML (Fawzi et al., 2022). Large structures will need developmental/generative encodings.

Availability

The SKI evaluator, parallel GA engine, CPU baseline, and benchmarks are open and reproducible on **Manhattan Reasoning Gym** (Barnard PL Labs, 2026): submit a design, program a cloud FPGA, and drive the search with a few register writes — inviting others to evolve in fabric.

References

- Barnard PL Labs (2026). Manhattan reasoning gym: a cloud FPGA platform for hardware-accelerated search. <https://manhattanreasoning.com>.
- Fawzi, A., Balog, M., Huang, A., et al. (2022). Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610:47–53.
- Fontana, W. and Buss, L. W. (1994). “the arrival of the fittest”: Toward a theory of biological organization. *Bulletin of Mathematical Biology*, 56(1):1–64.
- Miller, J. F., editor (2011). *Cartesian Genetic Programming*. Natural Computing Series. Springer.
- Mouret, J.-B. and Clune, J. (2015). Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*.
- Ray, T. S. (1991). An approach to the synthesis of life. In Langton, C. G. et al., editors, *Artificial Life II*, pages 371–408. Addison-Wesley.
- Thompson, A. (1996). An evolved circuit, intrinsic in silicon, entwined with physics. In *Evolvable Systems: From Biology to Hardware (ICES)*, volume 1259 of *LNCS*, pages 390–405. Springer.
- Vašíček, Z. and Sekanina, L. (2015). Evolutionary approach to approximate digital circuits design. *IEEE Transactions on Evolutionary Computation*, 19(3):432–444.